

Introduction to LEAN

Kovács György

Lean Summer School

2026

Table of Contents

1. The Entscheidungsproblem

2. Lambda Calculus

2.1 Untyped Lambda Calculus (λ)

2.2 Strictly Typed Lambda Calculus ($\lambda \rightarrow$)

2.3 System F ($\lambda 2$)

2.4 Lambda Omega ($\lambda\omega$)

2.5 Lambda P ($\lambda\Pi$)

3. Lean

3.1 What is LEAN?

3.2 Compilation

3.3 Types

3.4 Functions

3.5 Implicit Arguments

3.6 Typeclasses

3.7 Recursion

3.8 Pattern Matching

3.9 Dependent Types

3.10 Proofs

The Entscheidungsproblem

The Entscheidungsproblem (German for 'decision problem') is a challenge posed by David Hilbert and Wilhelm Ackermann in 1928. It asks for an algorithm that considers an input statement and answers "yes" or "no" according to whether it is universally valid.



Such an algorithm was proven to be impossible by Alonzo Church and Alan Turing in 1936.



Lambda Calculus

Lambda Expressions Definition

$$t := x \mid (\lambda x. t) \mid t \ t$$

where

t a **term**, defined recursively by this grammar

x a **variable**, the simplest kind of term

$(\lambda x. t)$ an **abstraction**: a function that takes a variable x and returns the term t

$t \ t$ an **application**: one term applied to another

Lambda Expressions Examples

$t ::= x \mid (\lambda x. t) \mid t \ t$

a just a variable

b just a variable

c just a variable

($\lambda a. a$) abstraction

($\lambda a. b$) abstraction

a b application

($\lambda x. x$) b application, first term of application is abstraction

($\lambda f. f \ f$) ($\lambda x. x$) application with first term being an abstraction that has an application in the body, and the second term of the application is another abstraction

Lambda Expressions Complex Examples

$a\ b\ c$ is an application, where the first term is an application $(a\ b)$, and the second term is a variable c . A clearer way to write this would be $(a\ b)\ c$. We will omit the parenthesis because they add too much noise

$(\lambda a.\lambda b.a\ b)$ This is an abstraction with multiple inputs, this is just a simplified notation, the more rigorous version would be $(\lambda a.(\lambda b.f\ a\ b))$

Lambda Calculus: Beta Reduction

$a \rightarrow a$

$a\ b \rightarrow a\ b$

Lambda Calculus: Beta Reduction

$a \rightarrow a$

$a \ b \rightarrow a \ b$

$(\lambda e.e) \rightarrow (\lambda e.e)$

$(\lambda a.a) \ b \rightarrow b$

Lambda Calculus: Beta Reduction

$a \rightarrow a$

$a \ b \rightarrow a \ b$

$(\lambda e.e) \rightarrow (\lambda e.e)$

$(\lambda a.a) \ b \rightarrow b$

$(\lambda c.x) \ b \rightarrow x$

$(\lambda x.f \ x) \ b \rightarrow f \ b$

$(\lambda a.\lambda b.f \ a \ b) \ x \ y \rightarrow (\lambda b.f \ x \ b) \ y \rightarrow f \ x \ y$

$(\lambda x.x) \ (\lambda x.x) \rightarrow (\lambda x.x)$

$(\lambda y.y \ y \ y) \ a \rightarrow a \ a \ a$

Lambda Calculus: Boolean

true := $(\lambda t. \lambda f. t)$

false := $(\lambda t. \lambda f. f)$

Lambda Calculus: Boolean

$\text{true} := (\lambda t. \lambda f. t)$

$\text{false} := (\lambda t. \lambda f. f)$

$\text{not} := (\lambda b. b \text{ false true})$

Lambda Calculus: Boolean

```
true := (λt.λf.t)  
false := (λt.λf.f)
```

```
not := (λb.b false true)  
and := (λa.λb.a b false)
```

Lambda Calculus: Boolean

```
true := (λt.λf.t)  
false := (λt.λf.f)
```

```
not := (λb.b false true)  
and := (λa.λb.a b false)  
or := (λa.λb.a true b)
```

Lambda Calculus: Boolean

```
true := (λt.λf.t)  
false := (λt.λf.f)
```

```
not := (λb.b false true)  
and := (λa.λb.a b false)  
or := (λa.λb.a true b)  
xor := (λa.λb.a b (not b))
```

Lambda Calculus: Boolean

$\text{true} := (\lambda t. \lambda f. t)$

$\text{false} := (\lambda t. \lambda f. f)$

$\text{not} := (\lambda b. b \text{ false } \text{true})$

$\text{and} := (\lambda a. \lambda b. a \ b \ \text{false})$

$\text{or} := (\lambda a. \lambda b. a \ \text{true} \ b)$

$\text{xor} := (\lambda a. \lambda b. a \ b \ (\text{not } b))$

$\text{if} := (\lambda c. \lambda t. \lambda f. c \ t \ f)$

Lambda Calculus: Natural Numbers

$0 := (\lambda s. \lambda z. z)$

$1 := (\lambda s. \lambda z. s \ z)$

$2 := (\lambda s. \lambda z. s \ (s \ z))$

$3 := (\lambda s. \lambda z. s \ (s \ (s \ z)))$

Lambda Calculus: Natural Numbers

$0 := (\lambda s. \lambda z. z)$

$1 := (\lambda s. \lambda z. s \ z)$

$2 := (\lambda s. \lambda z. s \ (s \ z))$

$3 := (\lambda s. \lambda z. s \ (s \ (s \ z)))$

$\text{succ} := (\lambda n. \lambda s. \lambda z. s \ (n \ s \ z))$

$\text{add} := (\lambda m. \lambda n. \lambda s. \lambda z. m \ s \ (n \ s \ z))$

$\text{mul} := (\lambda m. \lambda n. \lambda s. \lambda z. m \ (n \ s) \ z)$

$\text{isZero} := (\lambda n. n \ (\lambda x. \text{false}) \ \text{true})$

Lambda Calculus: Iteration

```
timesDoFor := ( $\lambda n. \lambda f. \lambda v. n \ f \ v$ )  
double := ( $\lambda n. \text{mul } n \ 2$ )  
timesDoFor 3 double 1  $\rightarrow$  8
```

Lambda Calculus: Recursion

$$Y := (\lambda f. (\lambda x. f \ (x \ x)))(\lambda x. f \ (x \ x))$$
$$Y \ f \rightarrow f \ (Y \ f) \rightarrow f \ (f \ (Y \ f)) \rightarrow \dots$$

Lambda Calculus: Nonsense

`isZero 0 → true`

`isZero 1 → false`

`isZero 2 → false`

`isZero 3 → false`

Lambda Calculus: Nonsense

`isZero 0 → true`

`isZero 1 → false`

`isZero 2 → false`

`isZero 3 → false`

`isZero false → true`

Lambda Calculus: Nonsense

```
isZero 0 → true  
isZero 1 → false  
isZero 2 → false  
isZero 3 → false  
isZero false → true  
isZero true → (λz.false)
```

Lambda Calculus: Nonsense

```
isZero 0 → true
isZero 1 → false
isZero 2 → false
isZero 3 → false
isZero false → true
isZero true → (λz.false)
isZero isZero → true
```

Lambda Calculus: Nonsense

```
isZero 0 → true  
isZero 1 → false  
isZero 2 → false  
isZero 3 → false  
isZero false → true  
isZero true → (λz.false)  
isZero isZero → true  
isZero (λa.a) → false
```

Strictly Typed Lambda Calculus

$$t := x \mid (\lambda x : T. t) \mid t \ t \mid c$$

$$T := \tau \mid T \rightarrow T$$

$(\lambda x : T. t)$ an **abstraction** which accepts objects of type T and returns t

c is a **constant**. We can introduce constants with specific types

T is a **type**, every object needs to have a specific type!

τ is a **type constant**, only types defined here will be in the language

Strictly Typed Lambda Calculus: Booleans

$$t := x \mid (\lambda x : T. t) \mid t \ t \mid \text{true} \mid \text{false} \mid \text{if } t \text{ then } t \text{ else } t$$
$$T := \text{Bool} \mid T \rightarrow T$$

In this language we have 3 constants:

1. $\text{true} : \text{Bool}$
2. $\text{false} : \text{Bool}$
3. $\text{if} : \text{Bool} \rightarrow \text{Bool} \rightarrow \text{Bool} \rightarrow \text{Bool}$

Strictly Typed Lambda Calculus: Booleans

$$t := x \mid (\lambda x : T. t) \mid t \ t \mid \text{true} \mid \text{false} \mid \text{if } t \text{ then } t \text{ else } t$$
$$T := \text{Bool} \mid T \rightarrow T$$

In this language we have 3 constants:

1. `true` : Bool
2. `false` : Bool
3. `if` : $\text{Bool} \rightarrow \text{Bool} \rightarrow \text{Bool} \rightarrow \text{Bool}$

Our language can have the following types:

- Bool
- $\text{Bool} \rightarrow \text{Bool}$
- $\text{Bool} \rightarrow \text{Bool} \rightarrow \text{Bool}$
- $(\text{Bool} \rightarrow \text{Bool}) \rightarrow \text{Bool}$
- $(\text{Bool} \rightarrow \text{Bool}) \rightarrow (\text{Bool} \rightarrow \text{Bool})$
- and so on

Strictly Typed Lambda Calculus: Booleans

if true then a else b $\rightarrow$ a
if false then a else b $\rightarrow$ b

Strictly Typed Lambda Calculus: Booleans

if true then a else b $\rightarrow$ a
if false then a else b $\rightarrow$ b

not : Bool $\rightarrow$ Bool
not := (λb : Bool. if b then false else true)

and : Bool $\rightarrow$ Bool $\rightarrow$ Bool
and := (λa : Bool. λb : Bool. if a then b else false)

or : Bool $\rightarrow$ Bool $\rightarrow$ Bool
or := (λa : Bool. λb : Bool. if a then true else b)

xor : Bool $\rightarrow$ Bool $\rightarrow$ Bool
xor := (λa : Bool. λb : Bool. if a then (not b) else b)

Strictly Typed Lambda Calculus: Natural Numbers

$$t := \dots \mid 0 \mid \text{succ } t \mid \text{rec } t \ t \ t$$
$$T := \text{Bool} \mid \text{Nat} \mid T \rightarrow T$$

- $0 : \text{Nat}$
- $\text{suc} : \text{Nat} \rightarrow \text{Nat}$
- $\text{rec} : \text{Nat} \rightarrow \tau \rightarrow (\text{Nat} \rightarrow \tau \rightarrow \tau) \rightarrow \tau$

Strictly Typed Lambda Calculus: Natural Numbers

$1 := \text{succ } 0$

$2 := \text{succ } 1$

$3 := \text{succ } 2$

$\dots$

$\text{add} : \text{Nat} \rightarrow \text{Nat} \rightarrow \text{Nat}$

$\text{add} := (\lambda a : \text{Nat} . \lambda b : \text{Nat} . ???)$

Strictly Typed Lambda Calculus: rec

$\text{rec} : \text{Nat} \rightarrow \text{Nat} \rightarrow (\text{Nat} \rightarrow \text{Nat} \rightarrow \text{Nat}) \rightarrow \text{Nat}$

$\text{rec } 3 \ z \ f$

$\rightarrow \text{rec } 2 \ (f \ 3 \ z) \ f$

$\rightarrow \text{rec } 1 \ (f \ 2 \ (f \ 3 \ z)) \ f$

$\rightarrow \text{rec } 0 \ (f \ 1 \ (f \ 2 \ (f \ 3 \ z))) \ f$

$\rightarrow f \ 1 \ (f \ 2 \ (f \ 3 \ z))$

$\text{add} : \text{Nat} \rightarrow \text{Nat} \rightarrow \text{Nat}$

$\text{add} := (\lambda a : \text{Nat} . \lambda b : \text{Nat} . \text{rec } a \ b \ (\lambda k : \text{Nat} . \lambda a : \text{Nat} . \text{succ } a))$

Strictly Typed Lambda Calculus: Addition

$f : \text{Nat} \rightarrow \text{Nat} \rightarrow \text{Nat}$

$f := (\lambda k : \text{Nat} . \lambda r : \text{Nat} . \text{succ } r)$

$\text{add} : \text{Nat} \rightarrow \text{Nat} \rightarrow \text{Nat}$

$\text{add} := (\lambda a : \text{Nat} . \lambda b : \text{Nat} . \text{rec } a \ b \ f)$

$\text{add } 2 \ 3$

$\rightarrow (\lambda a : \text{Nat} . \lambda b : \text{Nat} . \text{rec } a \ b \ f) \ 2 \ 3$

$\rightarrow \text{rec } 2 \ 3 \ f$

$\rightarrow \text{rec } 1 \ (f \ 2 \ 3) \ f$

$\rightarrow \text{rec } 0 \ (f \ 1 \ (f \ 2 \ 3)) \ f$

$\rightarrow f \ 1 \ (f \ 2 \ 3)$

$\rightarrow \text{succ } (\text{succ } 3)$

$\rightarrow \text{succ } 4$

$\rightarrow 5$

Strictly Typed Lambda Calculus: Other Operations

$\text{add} : \text{Nat} \rightarrow \text{Nat} \rightarrow \text{Nat}$

$\text{add} := (\lambda a : \text{Nat} . \lambda b : \text{Nat} . \text{rec } a \ b \ (\lambda k : \text{Nat} . \lambda r : \text{Nat} . \text{succ } r))$

$\text{mul} : \text{Nat} \rightarrow \text{Nat} \rightarrow \text{Nat}$

$\text{mul} := (\lambda a : \text{Nat} . \lambda b : \text{Nat} . \text{rec } a \ 0 \ (\lambda k : \text{Nat} . \lambda r : \text{Nat} . \text{add } r \ b))$

$\text{isZero} : \text{Nat} \rightarrow \text{Bool}$

$\text{isZero} := (\lambda n : \text{Nat} . \text{rec } n \ \text{true} \ (\lambda k : \text{Nat} . \lambda r : \text{Bool} . \text{false}))$

Strictly Typed Lambda Calculus: The Problem

$\text{add} : \text{Nat} \rightarrow \text{Nat} \rightarrow \text{Nat}$

$\text{add} := (\lambda a : \text{Nat} . \lambda b : \text{Nat} . \text{rec } a \ b \ (\lambda k : \text{Nat} . \lambda r : \text{Nat} . \text{succ } r))$

$\text{rec} : \text{Nat} \rightarrow \text{Nat} \rightarrow (\text{Nat} \rightarrow \text{Nat} \rightarrow \text{Nat}) \rightarrow \text{Nat}$

$\text{isZero} : \text{Nat} \rightarrow \text{Bool}$

$\text{isZero} := (\lambda n : \text{Nat} . \text{rec } n \ \text{true} \ (\lambda k : \text{Nat} . \lambda r : \text{Bool} . \text{false}))$

$\text{rec} : \text{Nat} \rightarrow \text{Bool} \rightarrow (\text{Nat} \rightarrow \text{Bool} \rightarrow \text{Bool}) \rightarrow \text{Bool}$

Every constant must have a single well defined type!

Strictly Typed Lambda Calculus: The Problem

`rec : Nat → Nat → (Nat → Nat → Nat) → Nat`

`rec : Nat → Bool → (Nat → Bool → Bool) → Bool`

`id : Bool → Bool`

`id : Nat → Nat`

`if : Bool → Bool → Bool → Bool`

`if : Bool → Nat → Nat → Nat`

`if : Bool → (Nat → Nat) → (Nat → Nat) → (Nat → Nat)`

`example := (if (isZero n) then succ else pred) 5`

Strictly Typed Lambda Calculus: Limitation

$\text{rec}_{\text{Nat}} : \text{Nat} \rightarrow \text{Nat} \rightarrow (\text{Nat} \rightarrow \text{Nat} \rightarrow \text{Nat}) \rightarrow \text{Nat}$

$\text{rec}_{\text{Bool}} : \text{Nat} \rightarrow \text{Bool} \rightarrow (\text{Nat} \rightarrow \text{Bool} \rightarrow \text{Bool}) \rightarrow \text{Bool}$

$\text{id}_{\text{Bool}} : \text{Bool} \rightarrow \text{Bool}$

$\text{id}_{\text{Nat}} : \text{Nat} \rightarrow \text{Nat}$

$\text{if}_{\text{Bool}} : \text{Bool} \rightarrow \text{Bool} \rightarrow \text{Bool} \rightarrow \text{Bool}$

$\text{if}_{\text{Nat}} : \text{Bool} \rightarrow \text{Nat} \rightarrow \text{Nat} \rightarrow \text{Nat}$

$\text{if}_{\text{Nat} \rightarrow \text{Nat}} : \text{Bool} \rightarrow (\text{Nat} \rightarrow \text{Nat}) \rightarrow (\text{Nat} \rightarrow \text{Nat}) \rightarrow (\text{Nat} \rightarrow \text{Nat})$

$\text{example} := (\text{if}_{\text{Nat} \rightarrow \text{Nat}} (\text{isZero } n) \text{ then succ else pred}) 5$

System F: Definitions

$$t := \dots \mid \Lambda A. t \mid t \{T\}$$
$$T := \text{Bool} \mid T \rightarrow T \mid A \mid \forall A. T$$
$$\text{id} : \forall A. A \rightarrow A$$
$$\text{id} := (\Lambda A. \lambda x : A. x)$$
$$\text{id}_{\text{Nat}} : \text{Nat} \rightarrow \text{Nat}$$
$$\text{id}_{\text{Nat}} := \text{id} \{ \text{Nat} \}$$
$$\text{id} \{ \text{Bool} \} \text{ true} \rightarrow \text{true}$$
$$\text{id} \{ \text{Nat} \} 3 \rightarrow 3$$
$$\text{id} \{ \text{Nat} \rightarrow \text{Bool} \} \text{ isZero} \rightarrow \text{isZero}$$

System F: We only need 1 of each function now

$\text{rec} : \forall T. \text{Nat} \rightarrow T \rightarrow (\text{Nat} \rightarrow T \rightarrow T) \rightarrow T$

$\text{id} : \forall T. T \rightarrow T$

$\text{if} : \forall T. \text{Bool} \rightarrow T \rightarrow T \rightarrow T$

This is equivalent to generics in languages such as Java.

```
class JavaExample {  
    <T> T id(T x) { return x; }  
    <T> T if(boolean b, T t, T f) {  
        return b ? t : f;  
    }  
}
```

System F: Complex Example

```
compose :  $\forall A. \forall B. \forall C. (A \rightarrow B) \rightarrow (B \rightarrow C) \rightarrow (A \rightarrow C)$   
compose := ( $\Lambda A. \Lambda B. \Lambda C. \lambda f_{ab} : A \rightarrow B. \lambda f_{bc} : B \rightarrow C. \lambda a :$   
A.  $f_{bc} (f_{ab} a)$ )
```

```
lessThan2 :  $\text{Nat} \rightarrow \text{Bool}$   
lessThan2 := compose {Nat} {Nat} {Bool} pred isZero
```

System F: Church Encodings are Back!

$\text{Bool} := \forall A. A \rightarrow A \rightarrow A$

$\text{true} := (\lambda A. \lambda t : A. \lambda f : A. t)$

$\text{false} := (\lambda A. \lambda t : A. \lambda f : A. f)$

$\text{not} : \text{Bool} \rightarrow \text{Bool}$

$\text{not} := (\lambda b : \text{Bool}. b \ \{\text{Bool}\} \ \text{false} \ \text{true})$

$\text{not} \ \text{false}$

System F: Church Encodings are Back!

$\text{Bool} := \forall A. A \rightarrow A \rightarrow A$

$\text{true} := (\lambda A. \lambda t : A. \lambda f : A. t)$

$\text{false} := (\lambda A. \lambda t : A. \lambda f : A. f)$

$\text{not} : \text{Bool} \rightarrow \text{Bool}$

$\text{not} := (\lambda b : \text{Bool}. b \ \{\text{Bool}\} \ \text{false} \ \text{true})$

$\text{not} \ \text{false}$

$\rightarrow (\lambda b : \text{Bool}. b \ \{\text{Bool}\} \ \text{false} \ \text{true}) \ \text{false}$

System F: Church Encodings are Back!

$\text{Bool} := \forall A. A \rightarrow A \rightarrow A$

$\text{true} := (\lambda A. \lambda t : A. \lambda f : A. t)$

$\text{false} := (\lambda A. \lambda t : A. \lambda f : A. f)$

$\text{not} : \text{Bool} \rightarrow \text{Bool}$

$\text{not} := (\lambda b : \text{Bool}. b \ \{\text{Bool}\} \ \text{false} \ \text{true})$

$\text{not} \ \text{false}$

$\rightarrow (\lambda b : \text{Bool}. b \ \{\text{Bool}\} \ \text{false} \ \text{true}) \ \text{false}$

$\rightarrow (\lambda A. \lambda t : A. \lambda f : A. f) \ \{\text{Bool}\} \ \text{false} \ \text{true}$

System F: Church Encodings are Back!

$\text{Bool} := \forall A. A \rightarrow A \rightarrow A$

$\text{true} := (\lambda A. \lambda t : A. \lambda f : A. t)$

$\text{false} := (\lambda A. \lambda t : A. \lambda f : A. f)$

$\text{not} : \text{Bool} \rightarrow \text{Bool}$

$\text{not} := (\lambda b : \text{Bool}. b \ \{\text{Bool}\} \ \text{false} \ \text{true})$

$\text{not} \ \text{false}$

$\rightarrow (\lambda b : \text{Bool}. b \ \{\text{Bool}\} \ \text{false} \ \text{true}) \ \text{false}$

$\rightarrow (\lambda A. \lambda t : A. \lambda f : A. f) \ \{\text{Bool}\} \ \text{false} \ \text{true}$

$\rightarrow (\lambda t : \text{Bool}. \lambda f : \text{Bool}. f) \ \text{false} \ \text{true}$

System F: Church Encodings are Back!

$\text{Bool} := \forall A. A \rightarrow A \rightarrow A$

$\text{true} := (\lambda A. \lambda t : A. \lambda f : A. t)$

$\text{false} := (\lambda A. \lambda t : A. \lambda f : A. f)$

$\text{not} : \text{Bool} \rightarrow \text{Bool}$

$\text{not} := (\lambda b : \text{Bool}. b \ \{\text{Bool}\} \ \text{false} \ \text{true})$

$\text{not} \ \text{false}$

$\rightarrow (\lambda b : \text{Bool}. b \ \{\text{Bool}\} \ \text{false} \ \text{true}) \ \text{false}$

$\rightarrow (\lambda A. \lambda t : A. \lambda f : A. f) \ \{\text{Bool}\} \ \text{false} \ \text{true}$

$\rightarrow (\lambda t : \text{Bool}. \lambda f : \text{Bool}. f) \ \text{false} \ \text{true}$

$\rightarrow (\lambda f : \text{Bool}. f) \ \text{true}$

System F: Church Encodings are Back!

$\text{Bool} := \forall A. A \rightarrow A \rightarrow A$

$\text{true} := (\lambda A. \lambda t : A. \lambda f : A. t)$

$\text{false} := (\lambda A. \lambda t : A. \lambda f : A. f)$

$\text{not} : \text{Bool} \rightarrow \text{Bool}$

$\text{not} := (\lambda b : \text{Bool}. b \ \{\text{Bool}\} \ \text{false} \ \text{true})$

$\text{not} \ \text{false}$

$\rightarrow (\lambda b : \text{Bool}. b \ \{\text{Bool}\} \ \text{false} \ \text{true}) \ \text{false}$

$\rightarrow (\lambda A. \lambda t : A. \lambda f : A. f) \ \{\text{Bool}\} \ \text{false} \ \text{true}$

$\rightarrow (\lambda t : \text{Bool}. \lambda f : \text{Bool}. f) \ \text{false} \ \text{true}$

$\rightarrow (\lambda f : \text{Bool}. f) \ \text{true}$

$\rightarrow \text{true}$

Since this is not how datatypes are implemented in LEAN, we will continue with the method from STLC and not mention this method again.

System F: Limitation

We might consider using types such as `List`, `Option`, `Tuple` and so on. While we can define such constructs, we can't really use them as first class citizens.

`Tuple A B :=  $\forall R. (A \rightarrow B \rightarrow R) \rightarrow R$`

We can defined `List of Nat` or `List of Bool`, but we can't talk generally about `List` or `Tuple`.

Lambda Omega: Definitions

$$K ::= \text{Type} \mid K \rightarrow K$$
$$T ::= \tau \mid T \rightarrow T \mid A \mid \forall A :: K. T \mid \lambda A :: K. T \mid T \ T$$
$$t ::= c \mid x \mid (\lambda x : T. t) \mid t \ t \mid (\lambda A :: K. t) \mid t \ \{T\}$$
$$\text{id} : \forall T :: \text{Type}. T \rightarrow T$$
$$\text{id} := (\lambda A :: \text{Type}. \lambda x : A. x)$$

Lambda Omega: Defining Option

$\text{Option} :: \text{Type} \rightarrow \text{Type}$

$\text{Option} := \lambda A :: \text{Type}. \forall R :: \text{Type}. R \rightarrow (A \rightarrow R) \rightarrow R$

$\text{none} : \forall A :: \text{Type}. \text{Option } A$

$\text{none} : (\Lambda A :: \text{Type}. \Lambda R :: \text{Type}. \lambda n : R. \lambda s : A \rightarrow R. n)$

$\text{some} : \forall A :: \text{Type}. A \rightarrow \text{Option } A$

$\text{some} := (\Lambda A :: \text{Type}. \lambda a : A. \Lambda R :: \text{Type}. \lambda n : R. \lambda s : A \rightarrow R. s \ a)$

$\text{map} : \forall A :: \text{Type}. \forall B :: \text{Type}. (A \rightarrow B) \rightarrow \text{Option } A \rightarrow \text{Option } B$

$\text{map} := \dots$

Lambda Omega: Higher Kinded Types

```
map : ∀F :: Type → Type. ∀A :: Type.∀B :: Type.(A → B) →  
F A → F B
```

We have the power to express higher kinded types from Haskell!

```
map : (a -> b) -> f a -> f b
```

Lambda Omega: Limitation

While we can define a variety of types, we can't define a type that depends on a term. The classic example is to have a Vector of exact length n .

```
Vector :: Nat → Type → Type  
example : Vector 5 Nat
```

Lambda Pi: Definition

$$T := \tau \mid T \rightarrow T \mid A \mid \forall A. T \mid \lambda A. T \mid T \ T \mid \Pi x : T. T \mid \text{Type}$$
$$t := c \mid x \mid (\lambda x : T. t) \mid t \ t \mid (\Lambda A : T. t) \mid t \ \{T\}$$

Kinds are merged into types, and a function is added that takes a term and produces a type.

Lambda Pi: Example

```
boolOrNat :  $\Pi x : \text{Bool}.$ if b then Bool else Nat  
boolOrNat := ( $\lambda b : \text{Bool}.$ if b then false else 0)
```

```
( $\Pi x : \text{Bool}.$ if b then Bool else Nat) false  
→ if false then Bool else Nat  
→ Nat
```

```
boolOrNat false  
→ ( $\lambda b : \text{Bool}.$ if b then false else 0) false  
→ if false then false else 0  
→ 0
```

Lambda Pi: Calculus of Constructions

We have reached the Calculus of Constructions. We are fully able to prove theorems with what we have. Now there are only a few steps until we reach LEAN.

$$(\lambda n : \text{Nat}. n + 0 = n)$$

We just need to add a couple of features, such as universes and a way to create types. We will not get too into those, since it is time to actually get introduced to LEAN.

`42 : Nat : Type : Type1 : Type2 : ...`

LEAN

What is LEAN?

Lean is an all purpose programming language designed by Leonardo de Moura and mantained by the Lean Focused Research Organization.

The first version of Lean was released in 2013, and the version that is used now (Lean 4) was released 10 years later.

Lean has been used to verify a number of theorems recently.

LEAN: Compilation and Elaboration

Lean is a compiled language. Here are the main steps:

1. **Parsing** Parses code into a syntax.
2. **Macro Expansion** Applies syntax changes.
3. **Elaboration** Transform syntax into code type theory.
4. **Kernel Checking** Verify if the result of elaboration follows the rules of type theory.
5. **Compilation** Compile to binary (C).

LEAN: Types

Lean comes with a number of predefined types such as `Nat`, `Int`, `String`, `List`, `Option` and so on.

Lean uses Algebraic Data Types (ADT), a way to build new types by combining existing types.

LEAN: Structures

```
structure Person where
```

```
  name : String
```

```
  age : Nat
```

```
def leo := Person.mk "Leonardo" 42 -- default constructor
```

```
def name := Person.name leo
```

```
def age := Person.age leo
```

```
-- evaluate the result and print
```

```
#eval leo -- { name := "Leonardo", age := 42 }
```

```
-- typecheck
```

```
#check leo -- leo : Person
```

```
#eval name -- "Leonardo"
```

```
#check name -- name : String
```

```
#eval age -- 42
```

```
#check age -- age : Nat
```

LEAN: Structures with Generics

```
structure MyProduct ( $\alpha$   $\beta$  : Type) where
  create :: -- custom constructor
  fst :  $\alpha$ 
  snd :  $\beta$ 

def tuple := MyProduct.create (23 : Nat) "lol"
def number := MyProduct.fst tuple
def string := MyProduct.snd tuple

#eval tuple -- { fst := 23, snd := "lol" }
#check tuple -- tuple : MyProduct Nat String
```

LEAN: Bool

```
inductive Boolean where  
  | true : Boolean  
  | false : Boolean
```

```
def isMyComputerOn := Boolean.true
```

```
#eval isMyComputerOn -- Boolean.true
```

```
#check isMyComputerOn -- isMyComputerOn : Boolean
```

LEAN: Unit

```
inductive MyUnit where
  | unit : MyUnit

def myUnit := MyUnit.unit

#eval myUnit -- Unit.unit
#check myUnit -- myUnit : MyUnit
```

LEAN: Void

```
inductive Void where
```

```
-- you can't have a value of type void because  
-- there are no constructors
```

LEAN: Option

```
inductive MyOption  $\alpha$  where
  | some :  $\alpha \rightarrow$  MyOption  $\alpha$ 
  | none : MyOption  $\alpha$ 

def something := some "Hello"
#check something -- Option String

def nothing : Option Nat := none
#check nothing -- Option Nat
```

LEAN: List

```
inductive MyList  $\alpha$  where
```

```
| cons :  $\alpha \rightarrow \text{MyList } \alpha \rightarrow \text{MyList } \alpha$ 
```

```
| nil : MyList  $\alpha$ 
```

```
def emptyList : MyList String := MyList.nil
```

```
def oneElement := MyList.cons 1 MyList.nil
```

```
def twoElements := MyList.cons 2 (MyList.cons 1  
MyList.nil)
```

-- Lean lists have sugaring for lists, so they look like
this:

```
def realEmpty : List String := []
```

```
def realElements := [1, 2, 3, 4]
```

LEAN: Functions

```
def myId (x :  $\alpha$ ) :  $\alpha$  := x
#eval myId 123 -- 123
```

```
def add1 (x : Nat) : Nat := x + 1
#eval add1 2 -- 3
```

```
def add (a : Nat) (b : Nat) : Nat := a + b
#eval add 5 2 -- 7
```

```
def myNot (b : Bool) : Bool := if b then false else true
#eval myNot true -- false
```

```
def myFunc (n : Nat) (f : Nat -> Nat) (g : Nat -> Nat) :
(Nat -> Nat) :=
  if n == 0 then f else g
```

```
#eval (myFunc 3 Nat.succ Nat.pred) 4 -- 3
```

LEAN: Lambda Functions

```
def add1 : Nat → Nat := λ x => x + 1
```

```
#eval (λ x => x) 5 -- 5
```

```
#eval List.map (λ n => n * n) [1, 2, 3] -- [1, 4, 9]
```

```
#eval List.filter (λ e => e % 2 == 0) [1,2,3,4,5,6]  
-- [2, 4, 6]
```

LEAN: Implicit Arguments

-- explicit: we must supply the type ourselves

```
def myId1 (T : Type) (x : T) : T := x
```

```
#eval myId1 Nat 5 -- 5
```

-- implicit: Lean infers it from the argument

```
def myId2 {T : Type} (x : T) : T := x
```

```
#eval myId2 5      -- 5, T := Nat inferred
```

```
#eval myId2 "hi"   -- "hi", T := String inferred
```

-- force it to be explicit anyway with @

```
#eval @myId2 Nat 5 -- 5
```

LEAN: Typeclasses

```
class Greet ( $\alpha$  : Type) where  
  greet :  $\alpha$   $\rightarrow$  String
```

```
instance : Greet Nat where  
  greet n := s!"Hello, number {n}!"
```

```
instance : Greet Bool where  
  greet b := if b then "Hello, true!" else "Hello,  
false!"
```

```
def greetIt { $\alpha$  : Type} [Greet  $\alpha$ ] (x :  $\alpha$ ) : String :=  
  Greet.greet x
```

```
#eval greetIt (5 : Nat)    -- "Hello, number 5!"  
#eval greetIt true        -- "Hello, true!"
```

LEAN: Recursion

```
def factorial (n : Nat) : Nat :=  
  if n == 0 then 1 else n * factorial (n - 1)
```

```
#eval factorial 5 -- 120
```

```
def length (xs : List  $\alpha$ ) : Nat :=  
  match xs with  
  | [] => 0  
  | _ :: rest => 1 + length rest
```

```
#eval length [1, 2, 3, 4] -- 4
```

Lean checks that recursive functions always terminate.

LEAN: Pattern Matching

```
def describe (n : Nat) : String :=  
  match n with  
  | 0 => "zero"  
  | 1 => "one"  
  | _ => "many"
```

```
#eval describe 7 -- "many"
```

```
def firstTwo (xs : List Nat) : String :=  
  match xs with  
  | [] => "empty"  
  | [x] => s!"one element: {x}"  
  | x :: y :: _ => s!"starts with {x}, {y}"
```

```
#eval firstTwo [1, 2, 3] -- "starts with 1, 2"
```

```
def safeDiv (a b : Nat) : Option Nat :=  
  match b with  
  | 0 => none  
  | _ => some (a / b)
```

LEAN: Dependent Types

```
def boolOrNat : (b : Bool) → (λ x => if x then Bool  
else Nat) b  
| true  => false  
| false => (0 : Nat)
```

The return **type** itself depends on the **value** of `b`. This is the Π -type from the Lambda Pi section, put to work: Lean's type system is built directly on it.

LEAN: Propositions are Types

So far Lean has looked like an ordinary functional language: types, structures, functions, recursion.

But recall the Calculus of Constructions from earlier: a statement like $2+2=4$ is itself just a **type**, and a **proof** of it is just a **term** of that type.

This means Lean can check proofs exactly the way it checks any other program.

LEAN: A Proof of Equality

```
inductive Eq :  $\alpha \rightarrow \beta \rightarrow$  Prop where
  | refl : (a :  $\alpha$ )  $\rightarrow$  Eq a a

-- you can't have something like Eq 1 2 (1 = 2)
-- because it's impossible to construct

theorem trans (h_ab : Eq a b) (h_bc : Eq b c) : Eq a c :=
  match h_ab with
  | Eq.refl a => h_bc -- this is the branch where a = b
```

We will explore how (and how far) this goes in the next session.

Thank You!